FULL-STACK ANALYSIS & GLOBAL BENCHMARK

BusTrack

A comprehensive audit of the BusTrack Ethiopian bus station management platform, covering security vulnerabilities, architecture assessment, UI/UX evaluation, global competitive analysis, and a phased roadmap to launch readiness.

Prepared for: BusTrack Engineering Team

Date: July 2025

Classification: Internal - Confidential

Table of Contents

Chapter 1: Executive Summary

BusTrack is a bus station ticket booking and management system designed for the Ethiopian market, targeting five distinct operational roles: Ticketer, Cashier, Gateman, Manager, and Superadmin. The platform is built on a modern technology stack including Next.js 16, React 19, Tailwind CSS 4, Prisma ORM with PostgreSQL (Neon), and Socket.IO for real-time communication. The system has been localized for Ethiopia with ETB currency, Telebirr mobile money integration placeholders, +251 phone formats, and AA plate number conventions. This report presents the findings of a comprehensive full-stack audit conducted across every layer of the application, from the UI components and API routes through to the database schema and deployment configuration.

The audit was supplemented by extensive global research into bus station digitalization practices, examining platforms across Africa (Kenya, Nigeria), Asia (India), the Americas (Colombia, Brazil), and Europe. This research covered market-leading solutions such as RedBus (India, serving 80,000+ routes), Optibus (Israel/Global, AI-powered transport OS), EBIX ITMS (India, managing 55,000+ buses), and African platforms like BuuPass, LiyuBus, and Mengedegna. The bus dispatch management software market is valued at USD 2.4 billion in 2024 and projected to reach USD 5.7 billion by 2034 at a CAGR of 10.6%, underscoring the significant commercial opportunity that BusTrack is positioned to capture in the underserved Ethiopian market.

Key Findings at a Glance

The analysis reveals a platform with a well-designed visual interface that effectively demonstrates the concept of a modern bus station management system, but which has critical foundational gaps that must be addressed before any production deployment. The most severe findings relate to security: the system stores passwords in plaintext, has zero authentication on all 16 API endpoints, exposes database credentials in version control, and has no middleware for route protection or rate limiting. These are not minor issues; they represent fundamental architectural gaps that would make the system immediately vulnerable in a production environment.

Severity	Count	Examples
Critical (Security)	7	Plaintext passwords, zero auth, exposed DB creds, no middleware
High (Data Integrity)	6	Float for money, timezone issues, race conditions, no transactions
Medium (Architecture)	15+	Unused deps, fake data, weak types, no pagination, no validation
Low (Polish/UX)	9	Static notifications, hardcoded stats, dead code, no fonts loaded

Table 1: Issue severity distribution from the full-stack audit.

On the positive side, the UI design demonstrates strong visual sensibility with role-differentiated color coding, responsive layouts, and a clean dark-mode aesthetic. The Prisma schema is well-structured with appropriate enums, indexes, and unique constraints. The real-time architecture using Socket.IO shows the

right approach for a live operations environment. The gap between the current state and launch readiness is significant but well-understood, and this report provides a detailed, phased roadmap to close it within approximately 16 weeks of focused development effort.

Chapter 2: Architecture Audit

2.1 Technology Stack Analysis

The BusTrack platform employs a deliberately modern technology stack that reflects current best practices in web development. Next.js 16.2 with App Router provides server-side rendering capabilities and a modern routing system, paired with React 19 for the component layer. The UI framework combines Tailwind CSS 4.3 with shadcn/ui components (new-york style), delivering a consistent design system with approximately 40 pre-built UI primitives. The data layer uses Prisma 6.11 with PostgreSQL via Neon serverless, and real-time communication is handled by Socket.IO 4.8 through a separate mini-service running on port 3004. The application is deployed as a standalone Next.js server output, with a Caddy reverse proxy and Docker support.

Layer	Technology	Version	Assessment
Frontend	Next.js + React	16.2 / 19	Bleeding edge, well-chosen
Styling	Tailwind CSS + shadcn/ui	4.3	Excellent, v4 CSS-first config
Database	PostgreSQL (Neon)	Prisma 6.11	Solid choice, serverless-ready
Real-time	Socket.IO	4.8.3	Correct architecture, separate service
State	React useState	N/A	Adequate for current scope
Deployment	Vercel + Docker + Caddy	Standalone	Good multi-option setup

Table 2: Current technology stack assessment.

2.2 Dependency Bloat Issue

A significant architectural concern is the presence of approximately 15 installed but completely unused dependencies. Packages including next-auth (v4.24), zustand (v5.0), @tanstack/react-query (v5.82), @tanstack/react-table (v8.21), react-hook-form, zod, @dnd-kit, date-fns, next-intl, and framer-motion are all listed in package.json but never imported anywhere in the codebase. This bloat increases the application bundle size, extends installation times, and creates confusion for developers joining the project who may assume these libraries are in active use. More critically, the presence of next-auth without any authentication implementation suggests an intent that was never executed, and the presence of zod without any validation schemas indicates a planned input validation layer that remains unbuilt. A dependency cleanup pass should be performed immediately, and the decision to remove or actually

implement these libraries should be documented in the project architecture records.

2.3 Database Schema Assessment

The Prisma schema defines 7 models (Staff, Station, Route, Bus, Schedule, Booking, Payment, GateLog) connected through well-structured relations with 6 enums covering roles, bus types, schedule statuses, booking statuses, payment methods, and gate validation results. The schema includes 11 database indexes on foreign keys and frequently queried fields, and critical unique constraints on email, plate number, booking reference, and the schedule-seat combination that prevents double-booking. The relational design is sound for a station-level management system, with appropriate many-to-one and one-to-many relationships that model the real-world domain accurately.

However, several data modeling issues were identified. The Payment model uses Float type for monetary values (fare, amount), which introduces floating-point rounding errors that are unacceptable for financial systems. The industry standard is to store monetary values as integers representing the smallest currency unit (cents) or to use the Decimal type. The Staff model has no soft-delete mechanism (deletedAt field), meaning all deletions would be permanent and irreversible. The stationId field on Staff is optional without a database-level constraint to enforce that SUPERADMIN users correctly have no station assignment. There are also two competing seed files (one for Addis Ababa, one for Nairobi) that create confusion about which data set is canonical, and both use the plaintext password "password" for all staff accounts.

2.4 API Architecture Review

The application exposes 16 API endpoints across authentication, bookings, payments, schedules, gate operations, dashboard statistics, and admin CRUD operations. The endpoints follow RESTful conventions with appropriate HTTP methods (GET for reads, POST for creates, PATCH for updates). The API structure maps logically to the five user roles, with each role having access to the endpoints relevant to its operational domain. However, the complete absence of any form of authentication, authorization, or rate limiting on every single endpoint means that any unauthenticated user can perform any operation, including creating staff accounts, modifying booking statuses, and accessing all dashboard analytics. The payment endpoint sets payment status to "COMPLETED" immediately without any actual payment gateway integration, making the entire financial flow simulated rather than real. Date filtering uses UTC-based timestamps which creates timezone misalignment for Ethiopian operations (UTC+3), potentially showing incorrect "today" data depending on when queries execute.

Category	Endpoints	Auth	Validation	Pagination
Authentication	1 (login)	None	None	N/A
Bookings	3 (CRUD)	None	None	Limit 50 only
Payments	2 (create, list)	None	None	Limit 20 only

Category	Endpoints	Auth	Validation	Pagination
Schedules/Routes	3 (list, today, detail)	None	None	None
Gate Operations	2 (validate, boarding)	None	None	N/A
Dashboard	2 (stats, departures)	None	None	None
Admin CRUD	4 pairs (GET/POST)	None	None	None

Table 3: API endpoint security and quality assessment.

Chapter 3: Security Assessment

The security posture of BusTrack in its current state is fundamentally insufficient for any production deployment. This chapter documents the critical vulnerabilities discovered during the audit, organized by severity. Each finding includes an explanation of the risk, the current state of the code, and the recommended remediation. It is important to note that these are not edge-case vulnerabilities; they are systemic architectural gaps that affect every aspect of the application. A production deployment of the current codebase would expose user data, financial records, and operational systems to significant risk.

3.1 Critical Vulnerabilities

3.1.1 Plaintext Password Storage

All staff passwords are stored in the PostgreSQL database as plaintext strings. The login endpoint (/api/auth/login) performs password comparison using a simple string equality check (=== operator). This means that anyone with database access, including through a SQL injection vulnerability or a compromised backup, would immediately have access to every user password in the system. The industry standard for password storage is to use a keyed hashing algorithm such as bcrypt, argon2, or scrypt, which are specifically designed to be computationally expensive and resistant to brute-force attacks. The remediation requires migrating all existing password hashes to bcrypt using a minimum work factor of 12, updating the login endpoint to use bcrypt.compare(), and implementing a password migration strategy for existing plaintext passwords.

3.1.2 Exposed Database Credentials

The Neon PostgreSQL connection string, including the username and password, is stored in the .env file at the project root. There is no .env.example file to document required environment variables, and the .env file appears to be committed to version control based on the audit findings. This means that anyone with access to the GitHub repository (L3von36/bustrack) has direct access to the production database. The immediate remediation is to rotate the database credentials, add .env to .gitignore, create a .env.example file with placeholder values, and configure all secrets through Vercel environment variables for the deployment environment.

3.1.3 Zero Authentication on API Endpoints

Not a single one of the 16 API endpoints implements any form of authentication or authorization. There is no middleware.ts file in the project, no JWT token validation, no session management, and no API key requirements. The client-side "authentication" is purely cosmetic, storing user data in React useState which is lost on every page refresh. The next-auth package is installed in package.json but never imported or configured anywhere. The admin endpoints for creating staff, routes, and buses are completely unprotected, meaning anyone on the internet can create admin accounts, modify routes, or add buses to the system. This is the single most critical issue that must be resolved before any form of production deployment, including beta testing with real users.

3.1.4 No Rate Limiting or CSRF Protection

The absence of rate limiting means that any endpoint can be called an unlimited number of times, enabling brute-force attacks on the login endpoint, denial-of-service through repeated expensive database queries, and automated scraping of all system data. There is no CSRF protection on any state-changing endpoint (POST, PATCH), which is particularly dangerous for a system that handles financial transactions. The Socket.IO server also has no authentication, with CORS set to wildcard origin ("*"), allowing any website to connect to the real-time event system and join any room including dashboard and gate operation rooms. This could allow malicious actors to inject fake booking events, manipulate departure boards, or disrupt gate scanning operations.

Chapter 4: UI/UX and Global Benchmark

4.1 Current Dashboard Evaluation

The BusTrack dashboard system serves five distinct roles through a single-page application pattern. Each role interface is dynamically imported with SSR disabled, and all share a common AppHeader component with role-colored pill badges, connection status indicator, notification bell, and user avatar with initials. The visual design demonstrates strong competency in modern dashboard aesthetics: the dark mode default with emerald accent colors creates a professional, operations-center feel. The Ticketer interface features a well-designed interactive seat map with visual differentiation between available, occupied, and selected seats. The Cashier interface uses an effective revenue hero section with large ETB currency formatting. The Gateman interface has a kiosk-style scan terminal with a radial emerald glow effect that is visually distinctive and functionally clear.

However, several UI/UX issues were identified through heuristic evaluation against established dashboard design principles. The Manager interface includes "AI Insights" that are entirely static and hardcoded, with no actual AI integration. The KPI "change" percentages (e.g., "+8%", "-1%") shown on dashboard cards are also hardcoded rather than calculated from actual data. The notification system shows a static count of "3" with three hardcoded notification items, and the "Mark all read" link has no

click handler. The Gateman interface uses a fallback manifest of 21 hardcoded Ethiopian names instead of real boarding data from the database. The Superadmin interface, at 1,347 lines, is the largest component and lacks edit/delete functionality for any managed entity (routes, buses, staff). The login screen has no password input field, always sending the hardcoded string "password" with every login attempt, which would be confusing for real users.

4.2 Global Platform Comparison

To contextualize BusTrack within the global landscape, the research examined major platforms across multiple continents. The comparison reveals that BusTrack is conceptually aligned with the right approach (role-based dashboards, real-time updates, seat selection) but lacks the depth of features, integration breadth, and operational maturity of established platforms. The key differentiator for BusTrack is its focus on station-level operations rather than passenger-facing booking, which is a relatively underserved niche globally. Most platforms focus on the B2C booking experience, while BusTrack targets the B2B operational management layer that actually runs the station.

Platform	Region	Scale	Key Differentiator
RedBus	India	80K+ routes, #1 booking	Dual B2B/B2C, polyglot microservices
EBIX ITMS	India	55K+ buses, 80K ETMs	Only fully integrated cloud ticketing
Optibus	Global	2,000+ cities	AI-powered planning + operations OS
Busbud	N. America	80+ countries	3,900+ operators, 1.4M+ routes
BuuPass	Kenya	East Africa	Mobile-first, M-Pesa integration
LiyuBus	Ethiopia	National	Leading local booking platform
BusTrack	Ethiopia	Single station	Station ops focus, 5-role system

Table 4: Global bus management platform comparison.

4.3 Dashboard Design Best Practices from Industry Research

Research into 2025 dashboard design principles reveals ten core tenets that should guide BusTrack evolution: user-primary focus (design for the specific operator, not generic dashboards), right visualizations (bar charts for comparisons, line charts for trends, gauges for KPIs), clear visual hierarchy (most important data at top-left following natural F-pattern scanning), reactive interactivity (every data element should be clickable for drill-down), responsive design (70%+ of African internet traffic is mobile), consistency (same color for same meaning across all dashboards), dark mode accessibility (already implemented), real-time speed (data should update within 3 seconds), personalization (users should customize their dashboard layout), and minimalism (maximum data density without overwhelming the user). The current BusTrack implementation follows approximately 6 of these 10 principles, with the gaps primarily in interactivity (no drill-down), personalization (fixed layouts), and real-time completeness

(some data is static/hardcoded).

4.4 Data Visualization Gaps

The current system uses Recharts only in the Superadmin Analytics tab, with a bar chart for revenue by route and a pie chart for passenger distribution. Industry-standard transport dashboards typically include sparkline KPIs showing trend direction, real-time fleet tracking maps, hourly passenger flow heatmaps, seat utilization heatmaps per route, revenue trend lines with comparison periods, and on-time performance gauges. The Manager dashboard includes SVG sparklines with Catmull-Rom to Bezier conversion for smooth curves, which is a positive indicator, but the sparkline data is generated from hardcoded arrays rather than actual historical data. To match industry standards, BusTrack needs to implement a time-series data collection mechanism (storing hourly/daily KPI snapshots) and connect all dashboard visualizations to real computed data rather than static placeholders.

Chapter 5: Ethiopian Market Analysis

5.1 Competitive Landscape

The Ethiopian bus transport digitalization market is in its early stages, presenting both a significant opportunity and a set of unique challenges. The primary existing platforms are LiyuBus, which serves as the leading Ethiopian online bus booking platform, and Mengedegna, the main mobile application for transport information. However, these platforms focus primarily on passenger-facing booking and information services, leaving the station operations management layer largely unaddressed. Anbessa City Bus Service, the largest public bus operator in Addis Ababa with 1,239 buses, operates without any modern digital management system, highlighting the enormous gap in the market. The Federal Transport Authority (FTA) under the Ministry of Transport and Logistics has been pushing for digitalization, including computerized vehicle and driver registration, electronic speed governing systems, and electronic ticketing requirements, creating regulatory tailwinds that favor BusTrack adoption.

5.2 Payment Integration Requirements

The Ethiopian digital payment landscape is dominated by two major mobile money platforms. Telebirr, launched by Ethio Telecom, is the market leader and offers three integration methods: H5 C2B Web Payment (redirect-based for web businesses), In-App Digital Payment API (for mobile applications), and the Fabric Payment Gateway (for backend enterprise systems). Integration requires RSA encryption for request signing, JSON RESTful endpoints, and webhook callbacks for payment status notifications. Merchant credentials must be obtained directly from Telebirr, and the API documentation, while available, has accessibility challenges from outside Ethiopia. The second major payment platform is M-Pesa, which Safaricom launched in Ethiopia in August 2023 after receiving a Payment Instrument

Issuer License. M-Pesa Ethiopia is integrated with EthSwitch (the national payment network) and enables cross-border remittances from 50+ countries. Ethiopian Airlines has already integrated M-Pesa for ticket payments, validating its viability for transport applications. A third option, CBE Birr (Commercial Bank of Ethiopia digital wallet), adds further coverage. The recommended approach is a triple payment integration covering Telebirr, M-Pesa, and cash, which would cover approximately 95% of Ethiopian payment methods.

Payment Method	Market Share	Integration Complexity	API Availability
Telebirr	Dominant (Ethio Telecom)	High (RSA encryption)	Requires direct contact
M-Pesa	Growing (since Aug 2023)	Medium (Daraja 3.0)	Sandbox available
CBE Birr	Major (national bank)	Medium	Limited documentation
Cash	Still significant	Low (manual)	N/A
Card/QR	Emerging	Medium	Via aggregators

Table 5: Ethiopian payment method landscape.

5.3 Regulatory Compliance Requirements

Operating a bus station management system in Ethiopia requires compliance with several regulatory frameworks administered by the Federal Transport Authority. Key requirements include computerized vehicle registration and driver licensing systems, electronic speed governing (ESG) mandates for all commercial vehicles, tax reporting obligations to the Ethiopian Revenue Authority, and transport permit management. As the regulatory framework for digital transport systems continues to evolve, BusTrack should be designed with compliance extensibility in mind, allowing new reporting requirements to be added without architectural changes. Comparison with other African markets shows that Kenya has a more mature framework through NTSA (National Transport and Safety Authority) with digital PSV licenses and KRA PIN integration, while India has the most comprehensive system through Vahan (vehicle registration), Sarathi (driver licensing), and GST reporting. BusTrack should target alignment with the most stringent requirements (India model) while implementing for the current Ethiopian regulatory environment.

5.4 Localization Requirements

Full Ethiopian localization extends beyond currency and phone format changes. The system needs native Amharic language support with Ge'ez script rendering (UTF-8), which requires proper font support and right-to-left awareness for Amharic text layout. The Ethiopian calendar system, which is approximately 7-8 years behind the Gregorian calendar and has 13 months (12 months of 30 days plus a 13th month of 5 or 6 days), should be supported as an alternative calendar view. Amharic uses a subject-object-verb (SOV) sentence structure that differs from English, requiring careful attention to UI text layout and string formatting. The i18next library, already installed but unused, provides the best-in-class

internationalization framework with ICU message format support that can handle these complexities. Additionally, offline-first capabilities are critical for the Ethiopian context where internet connectivity can be intermittent, particularly outside Addis Ababa.

Chapter 6: Feature Gap Analysis

6.1 Missing Critical Features

Comparing BusTrack against the feature sets of global platforms (RedBus, Optibus, EBIX ITMS, BuuPass) and the specific requirements of the Ethiopian market reveals significant feature gaps across multiple categories. The most critical missing features are those that directly impact core operations: real payment gateway integration (Telebirr/M-Pesa), proper authentication and session management, input validation across all forms and API endpoints, and data persistence for real-time events (the Socket.IO server stores all state in memory, meaning everything is lost on restart). Secondary but still important missing features include GPS-based fleet tracking, digital passenger manifests, luggage management, multi-station support, and an offline-first architecture for intermittent connectivity scenarios.

Feature	BusTrack	Industry Standard	Priority
Real Payment Integration	Simulated 2s timeout	Telebirr/M-Pesa/CBE Birr	P0 - Critical
Authentication System	None (plasma text)	JWT/OAuth2/SSO	P0 - Critical
Input Validation	None (Zod unused)	Server + client validation	P0 - Critical
GPS Fleet Tracking	Not implemented	Real-time map + geofencing	P0 - Critical
Amharic Localization	English only	Full i18n + Ethiopian calendar	P0 - Critical
Offline Capabilities	None	PWA + IndexedDB + sync	P1 - High
Digital Manifests	Fake data hardcoded	Real-time passenger lists	P1 - High
Revenue Reconciliation	Basic stats only	Full audit trail + reports	P1 - High
Luggage Management	Not implemented	Tag + track + reconcile	P2 - Medium
Dynamic Pricing	Not implemented	Demand + time-based fares	P2 - Medium
Multi-Station Support	Single station	Multi-tenant architecture	P2 - Medium
AI Route Optimization	Static hardcoded	ML-powered demand prediction	P3 - Low

Feature	BusTrack	Industry Standard	Priority
Loyalty Program	Not implemented	Points + tier rewards	P3 - Low

Table 6: Feature gap analysis against industry standards.

6.2 Competitor Feature Comparison Matrix

To understand where BusTrack stands relative to its competitive set, a feature comparison was conducted across the platforms most likely to compete for or complement BusTrack in the Ethiopian market. RedBus represents the global gold standard with its polyglot microservices architecture on AWS, supporting 80,000+ routes with dual B2B/B2C models. EBIX ITMS shows what is possible with deep government integration, managing 55,000+ buses with 80,000+ Android Electronic Ticketing Machines across India. Optibus demonstrates the power of AI-driven planning, offering end-to-end transport operations from schedule planning through real-time operations. Within Africa, BuuPass in Kenya shows the mobile-first approach that works for the continent, with deep M-Pesa integration and a focus on search-compare-book workflows. LiyuBus and Mengedegna in Ethiopia currently occupy the passenger-facing booking space, leaving the station operations layer that BusTrack targets largely uncontested.

Chapter 7: Launch Readiness Score

To provide a quantitative assessment of BusTrack current state, a launch readiness scoring model was developed covering 8 critical dimensions. Each dimension is scored on a 0-10 scale based on the audit findings, with weighted importance reflecting the relative impact on production viability. The overall weighted score provides a single metric that can be tracked over time as improvements are implemented. The current overall score of 2.6 out of 10 indicates that the platform is in an early prototype stage with a well-designed visual layer but critical foundational gaps.

Dimension	Weight	Score (0-10)	Assessment
Security & Auth	25%	0.5	Plaintext passwords, zero API auth, exposed creds
Data Integrity	15%	3.0	Good schema design but Float for money, race conditions
API Quality	15%	2.0	No validation, no pagination, no error handling
UI/UX Design	10%	7.0	Strong visual design, role-differentiated, responsive
Feature Completeness	10%	2.5	Core flow works, but all payments fake, no GPS
Code Quality	10%	3.5	Large components, unused deps, weak types, no tests
Operational Readiness	10%	2.0	No monitoring, no logging, no error boundaries
Localization	5%	4.0	ETB/phone/plate done, but no Amharic, no calendar

Table 7: Launch readiness scoring by dimension (weighted overall: 2.6/10).

7.1 Critical Blockers to Launch

Based on the scoring model and audit findings, the following items are absolute blockers that must be resolved before any form of production deployment, including closed beta testing. First, a complete authentication and authorization system must be implemented, covering JWT-based session management, role-based access control for all API endpoints, and proper password hashing with bcrypt. Second, all API endpoints must implement input validation using the already-installed Zod library, with server-side validation schemas for every request body. Third, the database credentials must be rotated and moved to environment variables that are not committed to version control. Fourth, real payment gateway integration must replace the simulated timeout, starting with at least Telebirr for the Ethiopian market. Fifth, monetary values must be migrated from Float to Integer (cents) or Decimal throughout the schema and all related code. These five items represent the minimum viable security and data integrity requirements for any system handling financial transactions.

Chapter 8: Strategic Roadmap

The following roadmap provides a phased approach to transforming BusTrack from its current prototype state into a launch-ready product. The roadmap is organized into four phases spanning approximately 16 weeks, prioritized by impact and dependency order. Each phase builds on the previous one, with Phase 1 addressing the most critical security and foundational issues, Phase 2 implementing core features that make the system functionally complete, Phase 3 adding advanced features that differentiate BusTrack from competitors, and Phase 4 focusing on scale, polish, and launch preparation. The estimated effort assumes a small team of 2-3 developers working full-time.

8.1 Phase 1: Security and Foundation (Weeks 1-2)

This phase addresses the five critical blockers identified in the launch readiness assessment. The authentication system should be built using NextAuth.js (already installed) with JWT-based sessions, configuring the CredentialsProvider to authenticate against the existing Staff model. Password hashing must be migrated to bcrypt with a minimum work factor of 12, and a migration script should update all existing plaintext passwords. Middleware must be created to protect all API routes, validating JWT tokens and enforcing role-based access control (e.g., only MANAGER and SUPERADMIN can access /api/admin/*, only GATEMAN can access /api/gate/*). Zod validation schemas must be defined for every API endpoint request body, and the Prisma schema must migrate monetary fields from Float to Int (representing cents). Database credentials must be rotated, .env added to .gitignore, and a .env.example file created. This phase also includes a dependency cleanup to remove the approximately 15 unused packages from package.json.

Task	Effort	Dependencies
Implement NextAuth.js with JWT + Credentials provider	2 days	None
Bcrypt password hashing + migration script	0.5 days	Auth system
Create middleware.ts for route protection + RBAC	1 day	Auth system
Zod validation schemas for all 16 endpoints	2 days	None
Migrate Float to Int for monetary fields	1 day	None
Rotate DB creds, add .gitignore, create .env.example	0.5 days	None
Remove unused dependencies	0.5 days	None
Add rate limiting (express-rate-limit or similar)	1 day	Middleware

Table 8: Phase 1 task breakdown (estimated 8.5 developer-days).

8.2 Phase 2: Core Features (Weeks 3-6)

With the security foundation in place, Phase 2 focuses on making the system functionally complete by replacing all simulated and hardcoded data with real implementations. The top priority is real payment integration, starting with Telebirr H5 C2B Web Payment (the most common use case for a web-based dashboard system). The integration requires obtaining merchant credentials from Telebirr, implementing RSA encryption for request signing, creating the payment initiation and callback handling endpoints, and building proper payment status polling. The second priority is replacing all hardcoded and fake data throughout the dashboards: real notifications from the database, computed KPI change percentages from historical data, real passenger manifests from booking records, and actual AI insights generated from operational data patterns. The third priority is implementing proper error boundaries, loading states, and connection failure handling for the Socket.IO real-time system, including persisting activity feed data to PostgreSQL instead of the current in-memory storage.

8.3 Phase 3: Advanced Features (Weeks 7-12)

Phase 3 introduces the features that will differentiate BusTrack from competitors and position it as the leading station management platform in Ethiopia. The first feature is GPS-based fleet tracking, which requires integrating a GPS tracking device or smartphone-based tracking app with the dashboard to show real-time bus locations on a map. This can be implemented using Mapbox or Leaflet with OpenStreetMap for cost-effective mapping. The second feature is full Amharic localization using the already-installed next-intl library, including a complete translation of all UI strings, Ge'ez font support, and optional Ethiopian calendar integration. The third feature is an offline-first architecture using Service Workers, IndexedDB, and background sync to ensure the system continues to function during internet outages, which are common outside Addis Ababa. The fourth feature is a digital manifest module that provides real-time passenger check-in tracking, seat utilization analytics, and boarding completion reports. The

fifth feature is multi-station support with a multi-tenant architecture that allows BusTrack to scale from a single terminal to a nationwide network of stations.

Feature	Effort	Impact	Technology
GPS Fleet Tracking	10 days	Critical	Mapbox + MQTT + WebSocket
Amharic Localization	5 days	Critical	next-intl + Noto Sans Ethiopic
Offline-First PWA	8 days	High	Service Worker + IndexedDB
Digital Manifests	5 days	High	Prisma queries + PDF export
Multi-Station Support	8 days	High	Multi-tenant Prisma schema
Luggage Management	5 days	Medium	QR tags + weight tracking
Dynamic Pricing Engine	5 days	Medium	Demand algorithm + admin UI
Notification System	3 days	Medium	Push + in-app + SMS via Africa Talking

Table 9: Phase 3 advanced feature roadmap.

8.4 Phase 4: Scale and Launch (Weeks 13-16)

The final phase prepares BusTrack for production launch through performance optimization, monitoring infrastructure, comprehensive testing, and deployment hardening. Performance optimization includes implementing React Query (already installed) for efficient data fetching with caching and automatic refetching, adding proper pagination to all list endpoints, optimizing the Socket.IO server with Redis-backed state persistence, and implementing connection pooling for the Prisma database client. Monitoring infrastructure should include Grafana dashboards for system metrics, error tracking with a service like Sentry, and application-level logging with structured JSON logs. Testing should cover unit tests for all utility functions and API routes using Vitest, integration tests for the booking and payment flows, and end-to-end tests using Playwright for the critical user journeys across all five roles. The deployment configuration should be hardened with proper health check endpoints, graceful shutdown handling, database migration scripts, and a CI/CD pipeline that runs tests and linting before every deployment to Vercel.

8.5 Key Differentiators for the Ethiopian Market

Based on the global research and Ethiopian market analysis, six key differentiators emerge that can position BusTrack as the dominant station management platform. First, being the first truly offline-first bus station management system designed for intermittent connectivity from day one, rather than added as an afterthought. Second, native Amharic support with Ethiopian calendar integration, demonstrating deep understanding of the local market. Third, triple payment integration (Telebirr + M-Pesa + cash) covering approximately 95% of Ethiopian payment methods. Fourth, government compliance built into the platform from the start, with FTA reporting, ESG data export, and vehicle/driver registration integration.

Fifth, a station operations focus that goes beyond booking to cover platforms, queues, announcements, and comprehensive terminal management. Sixth, a scalable cloud-native architecture that can grow from a single station to a national network without architectural changes, using multi-tenant design patterns from the beginning.

Differentiator	Competitive Advantage	Implementation Phase
Offline-First Architecture	No competitor offers this in Ethiopia	Phase 3 (Weeks 9-10)
Native Amharic + Ethiopian Calendar	Deep local market understanding	Phase 3 (Weeks 7-8)
Triple Payment Integration	95%+ payment method coverage	Phase 2 (Weeks 3-4)
Government Compliance Built-In	Regulatory moat, FTA alignment	Phase 2 (Weeks 5-6)
Station Operations Focus	Underserved niche vs booking platforms	Existing (Phase 1+2)
Scalable Multi-Tenant Architecture	Single station to national network	Phase 3 (Weeks 11-12)

Table 10: Strategic differentiators and implementation timeline.

This report was compiled from a full-stack code audit of the BusTrack repository (L3von36/bustrack), complemented by 25+ web searches and 15+ detailed page reads covering global bus station management platforms, dashboard design principles, Ethiopian transport technology, and East African payment integration. All findings are based on the codebase state as of July 2025 and the publicly available information from the referenced sources at the time of research.